

STARBUCKS CAPSTONE PROJECT REPORT

MACHINE LEARNING ENGINEER NANODEGREE

ŁUKASZ KORONA

1. Defining the problem

The goal of the project was to come up with a solution to analyze the impact of campaigns on monetary spend of Starbucks customers. I was provided with several datasets, each containing a different portion of the overall data. The portfolio dataset contained the summary of the campaign offers, the profile dataset looked at customer data, while the transcript file had all the relevant information about both offers and transactions.

The main difficulty in preparing the solution was how the data was structured. Each offer had different parameters, including duration in days. However, the transcript data contained timestamps treated as hours. In order to solve the challenge, I had to non-trivially transform the available json files to have a clear dependent variable for my machine learning models.

The hourly spend would later prove to be the dependent variable I was looking for. As a result, the problem turned out to be a regression one. I could come up with a variety of different models, one of which would be a benchmark - for that, I settled for simple linear regression available in scikit-learn. I would compare this solution against other algorithms, some of which were available in scikit-learn as well (random forests) and some of which were not (LightGBM).

In the end, it turned out that the tree-based models performed slightly better than the simple linear regression. However, using hourly spend caused some additional challenges that would be best solved with a different, binary model. The overall approach will be described in detail in the next sections of this document.

2. Analyzing the problem

The first step was to get familiar with the available datasets. I started with the raw portfolio dataset, which can be seen on the next page. The portfolio dataset contained information about the reward and the difficulty of the offer (good as features for the dataset), as well as available communication channels, duration of the offer and its type.

	reward	channels	difficulty	duration	offer_type	id
0	10	[email, mobile, social]	10	7	bogo	ae264e3637204a6fb9bb56bc8210ddfd
1	10	[web, email, mobile, social]	10	5	bogo	4d5c57ea9a6940dd891ad53e9dbe8da0
2	0	[web, email, mobile]	0	4	informational	3f207df678b143eea3cee63160fa8bed
3	5	[web, email, mobile]	5	7	bogo	9b98b8c7a33c4b65b9aebfe6a799e6d9
4	5	[web, email]	20	10	discount	0b1e1539f2cc45b7b9fa7c272da2e1d7
5	3	[web, email, mobile, social]	7	7	discount	2298d6c36e964ae4a3e7e9706d1fb8c2
6	2	[web, email, mobile, social]	10	10	discount	fafdc668e3743c1bb461111dcafc2a4
7	0	[email, mobile, social]	0	3	informational	5a8bc65990b245e5a138643cd4eb9837
8	5	[web, email, mobile, social]	5	5	bogo	f19421c1d4aa40978ebb69ca19b0e20d
9	2	[web, email, mobile]	10	7	discount	2906b810c7d4411798c6938adc9daaa5

Next, I looked at the profile dataset, which contained age, gender and income of Starbucks customers. Unlike the portfolio dataset, it also contained some missing values.

	gender	age	id	became_member_on	income
0	None	118	68be06ca386d4c31939f3a4f0e3dd783	20170212	NaN
1	F	55	0610b486422d4921ae7d2bf64640c50b	20170715	112000.0
2	None	118	38fe809add3b4fcf9315a9694bb96ff5	20180712	NaN
3	F	75	78afa995795e4d85b5d9ceeca43f5fef	20170509	100000.0
4	None	118	a03223e636434f42ac4c3df47e8bac43	20170804	NaN

Finally, I looked at the transcript dataset.

15	f806632c011441378d4646567f357a21	offer received	{'offer id': 'fafdc668e3743c1bb461111dcafc2a4'}	0
16	d058f73bf8674a26a95227db098147b1	offer received	{'offer id': '0b1e1539f2cc45b7b9fa7c272da2e1d7'}	0
17	65aba5c617294649aeb624da249e1ee5	offer received	{'offer id': '2906b810c7d4411798c6938adc9daaa5'}	0
18	ebe7ef46ea6f4963a7dd49f501b26779	offer received	{'offer id': '9b98b8c7a33c4b65b9aebfe6a799e6d9'}	0
19	1e9420836d554513ab90eba98552d0a9	offer received	{'offer id': 'ae264e3637204a6fb9bb56bc8210ddfd'}	0

	person	event	value	time
306514	f1e4fd36e5a0446f83861308bddf6945	transaction	{'amount': 8.2}	714
306515	0b64be3b241c4407a5c9a71781173829	transaction	{'amount': 2.6}	714
306516	86d03d35d7e0434b935e7743e83be3a0	transaction	{'amount': 9.2}	714
306517	3408fd05c781401f8442fb6dbaaea9c7	transaction	{'amount': 11.7}	714

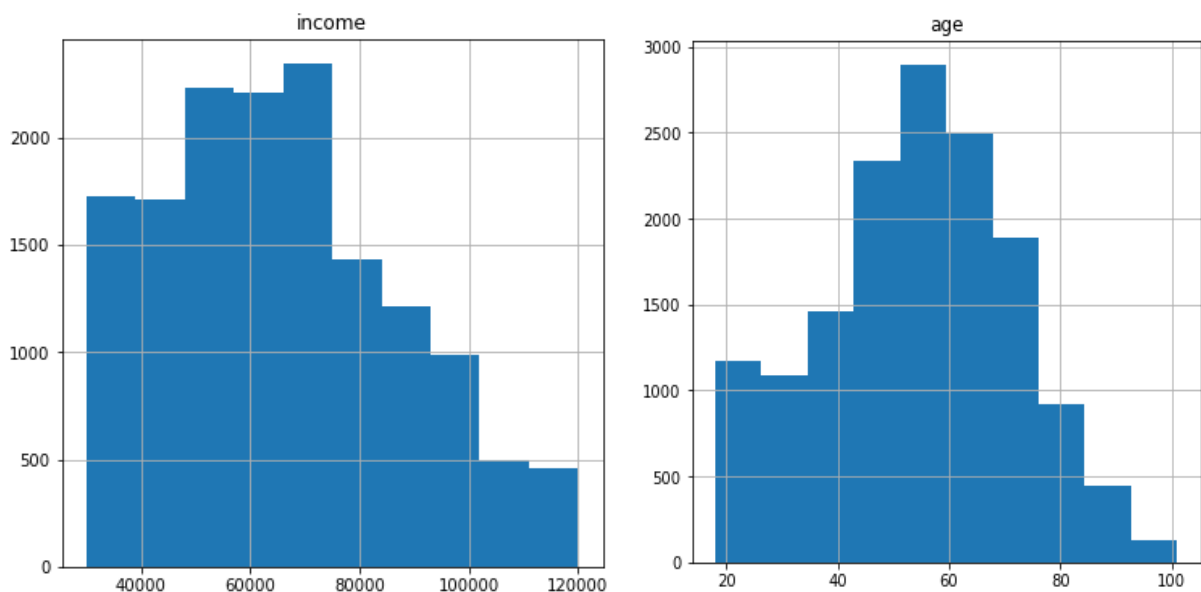
This dataset contained all the information about transactions and offers (whether they were received, viewed and/or completed). It also seemed like the part of the data that would be the hardest to process.

The second step towards analyzing the problem was actually described in the third section of this document (as data preprocessing was a necessary part to do the analysis). However, with a properly processed dataset I was able to do an exploratory data analysis that helped me confirm my suspicions as to what the correct direction was.

I started off with univariate analysis of certain variables. One of them was related to customer data (age). First of all, it turned out that some values were missing, but they were encoded as 118 (a pretty unusual age for a customer of any establishment). Moreover, customers with age equal to 118 were not described with gender and their income either. As a result, I decided to get rid of those entries in the profile dataframe. The chart below shows a sample from this dataset, with customers with missing values clearly visible.

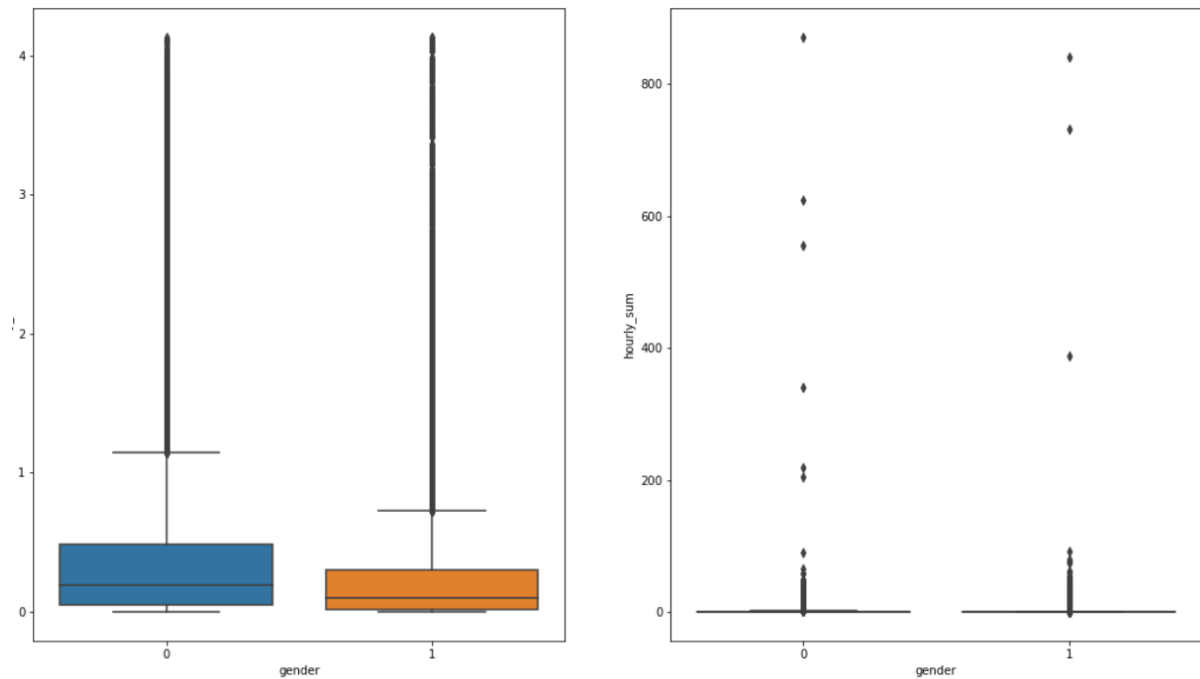
	gender	age	id	became_member_on	income
0	None	118	68be06ca386d4c31939f3a4f0e3dd783	20170212	NaN
1	F	55	0610b486422d4921ae7d2bf64640c50b	20170715	112000.0
2	None	118	38fe809add3b4fcf9315a9694bb96ff5	20180712	NaN
3	F	75	78afa995795e4d85b5d9ceeca43f5fef	20170509	100000.0
4	None	118	a03223e636434f42ac4c3df47e8bac43	20170804	NaN

The next step in analyzing the available data was to check the distributions of the available features from the profile dataset. That included age and income, histograms of which are shown below:

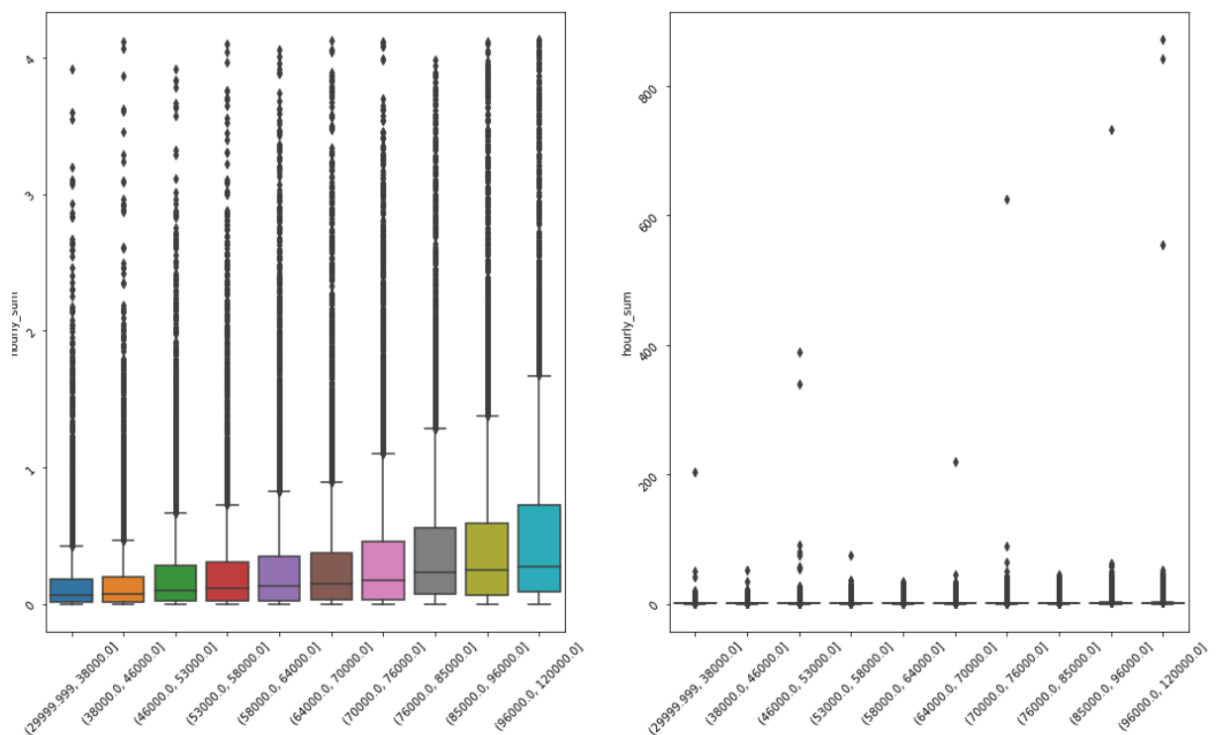


Neither distribution is normal, but they can be easily transformed (and for linear regression they actually have to be standardized).

I also checked how different features would be correlated with the hourly spend. For both the categorical and the numerical features I used boxplots (for numerical ones, I first binned the features into quantiles). One such boxplot (for gender) can be seen below.



The left side version of the chart is capped at the 95th percentile, while the right side version contains all data points. As we can see, there are a number of outliers for the dependent variable. Women also spend a bit more than men.



Another example of the boxplots is related to the income variable, which is shown on the previous page. The higher the income of the customer, the bigger their daily spend.

3. Implementation

The implementation section can be divided into three parts:

- preprocessing the data
- building a docker container
- training the models

3.1. Preprocessing

The raw data about customers and offers had to be first cleaned. Moreover, dummy variables had to be created for every categorical variable (for linear regression). Label encoding was used for categorical variables as this is how categoricals can be handled by LightGBM.

The next, much more intricate step, was to process the transcript data. After cleaning the columns of the json file, a new dataset had to be constructed, where each row would correspond to one, unduplicated (as in, running as the sole offer at a specific time) offer, along with its duration (from viewing till completion/end). Then, additional rows would be added for the sum of time ranges when no offer was running.

The process had to be run iteratively, for each customer, and each offer the customer viewed. It can be summarized with the following algorithm:

for each customer

for each received offer

- check if and when the offer was viewed (**must happen before completion/end**)
- check if the offer overlapped with any other viewed offer
- if overlap, move to the next iteration (offer)
- if viewed and no overlap
 - get the sum of transactions between viewing and end
 - get the sum of transactions when the customer was not aware of the offer
 - calculate time ranges

for timeranges without any **viewed** offer running

- calculate total money spent
- calculate time range without an offer

An extract of the dataframe can be seen in the screenshot below:

reward	difficulty	email	mobile	social	web	bogo	discount	informational	channels_int	offer_type_int	duration_h_y	gender	age	income	year	hourly_sum
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	999.0	999.0	0.0	1	33	72000.0	2017	0.159588
0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	1.0	2.0	2.0	96.0	1	33	72000.0	2017	0.142833
0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	1.0	0.0	2.0	72.0	1	33	72000.0	2017	0.461867
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	999.0	999.0	0.0	0	40	57000.0	2018	0.126685
3.0	7.0	1.0	1.0	1.0	1.0	0.0	1.0	0.0	3.0	1.0	168.0	0	40	57000.0	2018	0.178060

Not all of the columns would be used as features - some of them would only be used for linear regression, others only for LightGBM and RandomForests, and then some were just needed to get values of other columns.

3.2. Building docker container

While linear regression and random forests can now be used with SKLearn Sagemaker estimator, LightGBM, unlike its counterpart (XGBoost), is not available off-the-shelf. As a result, a Docker container had to be created so that training and inference with LightGBM could be done.

The container was based on Linux distribution Ubuntu 16.04. The dockerfile contained instructions to install python as well as other necessary software (nginx to be able to set up an endpoint). The packages necessary for training and inference were also added to the container (that included not just LightGBM, but also gunicorn and flask - for the endpoint).

The relevant train, serve and predictor scripts were copied within the container from the Notebook instance that I used for coding.

After the dockerfile was set up properly, the container was pushed to the Elastic Container Registry, and from there, could be invoked to set up both training and prediction jobs.

3.3. Training

Several different models were used for training. I started with simple linear regression with SKLearn Sagemaker estimator. An important part of training with simple linear regression was making sure that the numerical features were standardized to make the coefficients realistic and scale-invariant.

I did not have to scale the data when I proceeded to the random forests (also from SKLearn) and finally, LightGBM that I was able to train using the container described in the previous subsection. Both the random forests and LightGBM were however tuned on several

hyperparameters. For random forests, I chose the `min_samples_leaf` and `n_estimators`, while for LightGBM I looked at `max_depth` and `num_of_leaves`.

The metric that was used for minimizing the LightGBM algorithm was RMSE.

4. Results

The first experiment did not run smoothly - the metric chosen for validation (root mean squared error) was quite high (~8.42) and the model was constantly overpredicting the hourly spend. The problem most likely stemmed from the fact that the outliers were still present in the dataset.

After that, another experiment was run, this time with outliers excluded (anything over the 95th percentile was cut). RMSE on the validation set dropped dramatically (~0.533), which proved that it was a good direction.

The next experiments were run with the use of random forests and LightGBM. The performance on the validation set was consistently (for any set of hyperparameters) lower than for Linear Regression. However, it was not a very big difference, as RMSE ranged somewhere around ~0.512-0.514. Additional experiments, on the whole dataset, were run for tree-based models, and similarly, RMSE was lower, but not by much (~8.38). Note: the results of those experiments are not available in the project notebook, to make it more concise and easier to digest.

An additional exercise was done to check how we could pick up the best offer for a given customer/calculate the uplift of each offer. Code was written to pick up customers at random from either the training or validation dataset and then simulate what the hourly spend would be for each of the campaigns. A sample table is available below.

reward	difficulty	channels_int	offer_type_int	gender	age	income	year	hourly_predicted
10	10	0	0	0	75	100000.0	2017	0.851333
10	10	3	0	0	75	100000.0	2017	0.865740
0	0	2	2	0	75	100000.0	2017	0.314531
5	5	2	0	0	75	100000.0	2017	0.830446
5	20	1	1	0	75	100000.0	2017	0.851840
3	7	3	1	0	75	100000.0	2017	0.870468
2	10	3	1	0	75	100000.0	2017	0.871840
0	0	0	2	0	75	100000.0	2017	0.317760
5	5	3	0	0	75	100000.0	2017	0.864012
2	10	2	1	0	75	100000.0	2017	0.816603
0	0	999	999	0	75	100000.0	2017	0.214577

As we can see, the customer spends the most when the offer is a discount one without too much difficulty (10 or 7). Then, the average hourly spend is 0.87 cents. We can also see the uplift on the informative campaigns, which is about 10 cents higher than the baseline. Without the baseline, the customer spends on average 21 cents.

It is worth to note that this specific customer does not have a big difference in spend for any of the rewarded campaigns. Therefore, it might be more beneficial to go for less expensive (having a lower reward) campaigns than for the BOGO offers.

To give a counterexample, we can see another player's simulation below.

reward	difficulty	channels_int	offer_type_int	gender	age	income	year	hourly_predicted
10	10	0	0	1	68	70000.0	2018	0.216822
10	10	3	0	1	68	70000.0	2018	0.217526
0	0	2	2	1	68	70000.0	2018	0.104176
5	5	2	0	1	68	70000.0	2018	0.300197
5	20	1	1	1	68	70000.0	2018	0.366113
3	7	3	1	1	68	70000.0	2018	0.271602
2	10	3	1	1	68	70000.0	2018	0.294499
0	0	0	2	1	68	70000.0	2018	0.092322
5	5	3	0	1	68	70000.0	2018	0.234265
2	10	2	1	1	68	70000.0	2018	0.358298
0	0	999	999	1	68	70000.0	2018	0.081768

This customer likes spending with different offers (but still discount ones). He might not be too affected by the informative campaigns though.

5. Conclusions

The main aim of the project: to predict the spend of a player dependent on the offer was successfully done. A variety of methods were used to get the satisfying results, and that included simple linear regression, random forests and LightGBM.

Tree methods performed slightly better than linear regression, but RMSE could still be improved with more data (especially when the offer was running, as the data was a bit scarce on that).

There are still other shortcomings of the model. One of them is that outliers cannot be used for training, as they tend to cause overpredicting for non-outlier data points. However, another model could be constructed to get rid of that problem (for example, a model that detects which customers are most likely to react immediately to an offer, and with a massive spend).

Another shortcoming is that the model answers some questions only indirectly. One of those questions would be - is it better to give someone a BOGO offer or no offer at all (since the cost of the BOGO offer is equal to its difficulty). However, another model could also be constructed for hunting of such 'bonus' abusers. For all the other customers, it is still possible to assess which offer they should get - in case of similar hourly spend, always go for the cheapest offer!

The informative offers should also be treated separately, as they give a clear uplift, so as long as they are cheaper than, for a player in the example, 10 cents per hour, they should be running constantly for this player. That brings us to the third shortcoming of the model, one that goes beyond the available data, namely: at what point, enough is enough when it comes to sending out offers? Maybe some customer would not like to be constantly spammed with campaigns?